

HIGH-LEVEL DESIGN (HLD)

RAG-Based Customer Support Assistant

LangGraph | ChromaDB | OpenAI GPT-4o-mini | HITL | Innomatics Research Labs

Project	RAG-Based Customer Support Assistant
Version	1.0
Date	May 2026
Technology Stack	Python, LangChain, LangGraph, ChromaDB, OpenAI

1. System Overview

Problem Definition: Traditional customer support systems rely on static FAQs or rule-based chatbots that cannot handle dynamic, document-grounded queries. They hallucinate answers, miss context, and cannot escalate intelligently.

Solution: A Retrieval-Augmented Generation (RAG) system that reads a PDF knowledge base, retrieves the most relevant information using semantic search, and generates accurate, grounded answers. When confidence is low, it escalates to a human agent via HITL.

Scope:

- Load and index PDF documents into ChromaDB vector store
- Accept natural language user queries via CLI
- Retrieve semantically similar chunks using embeddings
- Generate answers using OpenAI GPT-4o-mini
- Route low-confidence queries to human agents (HITL)
- Graph-based control flow using LangGraph (2-node: process → output)

2. Architecture Diagram

The system is composed of the following layers:

Layer	Component	Description
Input	CLI Interface	User types question; system accepts text input
Ingestion	Document Ingestion Pipeline	PyPDFLoader loads PDF pages; metadata tagged per source
Chunking	RecursiveCharacterTextSplitter	Splits pages into 500-char chunks with 100-char overlap
Embedding	HuggingFace all-MiniLM-L6-v2	Converts text chunks to 384-dim dense vectors (free, local)

Vector Store	ChromaDB (persisted)	Stores and indexes embeddings; similarity search via cosine distance
Retrieval	similarity_search_with_score	Top-3 chunks retrieved; score threshold 1.5 for quality gate
LLM Layer	OpenAI GPT-4o-mini	Generates answer from retrieved context + user query
Workflow	LangGraph StateGraph	2-node graph: process_node → output_node with state passing
HITL	human_fallback()	Escalates to human when context not found or score too high

3. Component Description

Document Loader (ingest.py)

PyPDFLoader reads PDF files page by page. Metadata (source path) is attached to each document object for traceability.

Chunking Strategy (ingest.py)

RecursiveCharacterTextSplitter with chunk_size=500 and chunk_overlap=100. Overlap ensures no context is lost at chunk boundaries.

Embedding Model (ingest.py / retrieval.py)

HuggingFace all-MiniLM-L6-v2 — a lightweight, free, local model that produces 384-dimensional semantic embeddings.

Vector Store (ingest.py / retrieval.py)

ChromaDB persisted to ./db directory. Embeddings stored once; reused across sessions. Supports cosine similarity search.

Retriever (retrieval.py)

Fetches top-3 similar chunks. Applies score threshold (>1.5 = irrelevant). Filters headings and returns the most query-relevant line.

LLM (generator.py)

OpenAI GPT-4o-mini. Receives retrieved context + user query as a structured prompt. Temperature=0.3 for consistent, factual answers.

Graph Workflow Engine (graph.py)

LangGraph StateGraph with two nodes: process_node (retrieval + escalation check) and output_node (answer or HITL call).

Routing Layer (graph.py)

state['escalate'] flag controls routing. Set to True when no relevant chunk found or LLM returns None.

HITL Module (hitl.py)

human_fallback() prints the query and prompts a human operator via terminal input(). Returns the human-typed answer.

4. Data Flow

Ingestion Flow (runs once on first launch):

Step	Action	Output
1	PyPDFLoader reads PDF	List of Document objects with text + metadata
2	RecursiveCharacterTextSplitter splits pages	~N chunks of 500 chars each
3	HuggingFace encodes chunks	384-dim embedding vectors
4	ChromaDB stores vectors	Persisted ./db directory with index

Query Lifecycle (every user question):

Step	Action	Output
1	User types query in CLI	Raw text string
2	LangGraph invokes process_node	State dict populated
3	retrieve_docs() searches ChromaDB	Top-3 chunks with scores
4a	Score $\leq 1.5 \rightarrow$ generate_answer()	GPT-4o-mini answer string
4b	Score $> 1.5 \rightarrow$ escalate=True	HITL triggered
5	output_node prints final answer	AI or Human answer displayed

5. Technology Choices

ChromaDB: Open-source, local, zero-cost vector database. Persistent storage across sessions. Native LangChain integration. Simple Python API.

LangGraph: Graph-based workflow engine built on LangChain. Enables explicit control flow with stateful nodes and conditional edges. Better than simple chains for HITL logic.

all-MiniLM-L6-v2: Free HuggingFace model. 384-dim vectors. Fast, runs locally with no API cost. Excellent semantic similarity for English text.

GPT-4o-mini: Cost-efficient OpenAI model. Strong instruction-following for support-style Q&A.; Temperature 0.3 ensures factual, consistent answers.

LangChain: Provides document loaders, text splitters, and vector store wrappers that abstract complex retrieval logic into simple function calls.

6. Scalability Considerations

Handling Large Documents: Increase chunk_overlap for better context continuity. Use larger k in similarity_search. Consider sentence-level chunking for precision.

Increasing Query Load: ChromaDB can be replaced with Pinecone or Weaviate for distributed, cloud-hosted vector search at scale.

Latency Concerns: Cache frequently asked queries. Use streaming responses from OpenAI. Pre-warm the embedding model at startup.

Multi-Document Support: Already supported — `ingest_pdfs()` accepts a list of PDF paths. Metadata tagging tracks source per chunk.